

I need help figuring out the kinematics for a parallel axis tripteron mechanism. Are there any papers I can look at to begin?

Great question — the **Tripteron** is actually one of the *cleanest* parallel mechanisms to start learning kinematics on, especially if you're dealing with a **parallel-axis (Cartesian-style) architecture**.

Let me point you to **high-quality papers + how to read them**, and then I'll give you a **mental model** so you don't get lost in the math.

Core papers to start with

1. Foundational Tripteron kinematics/dynamics

- [Tripteron Parallel Robot \(Petkova 2022 PDF\)](#) ↗
 - Covers **direct + inverse kinematics explicitly**
 - Good entry-level derivations
 - Focus: 3-DOF translational Tripteron

👉 Why start here: it walks through solving both FK and IK for the exact mechanism you care about. AIP Publis...

2. Classic reference (Gosselin — highly cited)

- *Compact dynamic models for the Tripteron and quadrupteron*
 - Authors: Clément Gosselin et al.
 - Journal: Proc. IMechE (2009)

👉 Why this matters:

- Defines the **canonical Tripteron architecture**
 - Includes a **kinematics recap before dynamics**
 - This is the paper most others build on Sage Journals
-

3. Experimental / real-world kinematics issues

- *Experimental kinematic identification of a 3-DOF Tripteron*
 - Focus: calibration + model mismatch

👉 Why it's useful:

- Shows how **real kinematics deviate from theory**
 - Important if you're building hardware Sage Journals
-

4. Broader translational parallel robot context

- *Kinematics and workspace analysis of the Orthoglide*
 - Closely related architecture

👉 Why read this:

- The Orthoglide is basically a **Tripteron cousin**
 - Often **cleaner derivations + better geometric intuition** arXiv
-

5. General Tripteron modeling (dynamics-heavy but useful)

- *Modeling and control of a Tripteron*

👉 Useful for:

- Seeing how kinematics plugs into full system modeling ScienceDirect
-

Key insight (this will save you hours)

The Tripteron is a **Cartesian parallel manipulator**:

- 3 legs
- Each leg constrains motion along a direction
- End-effector = intersection of constraints

Structurally:

- Typically **3(PRRR)** chains Wikipedia
 - Each actuator produces **pure translation DOF**
-

How to think about the kinematics (important)

1. Inverse kinematics (easy)

This is the big win of the Tripteron:

👉 IK is usually **decoupled and linear**

For parallel-axis version:

$$x = f_1(q_1), \quad y = f_2(q_2), \quad z = f_3(q_3)$$

In many designs:

- Each actuator directly controls **one Cartesian direction**
 - So IK $\approx$ **projection onto actuator axis**
-

2. Forward kinematics (also easy here!)

Unlike most parallel robots:

👉 Tripteron FK is **NOT** nasty

Because:

- Motion is purely translational
- Chains are orthogonal / parallel

So FK reduces to solving a **3×3 linear system**, not nonlinear polynomials.

3. Constraint-based interpretation (very powerful)

Each leg imposes:

“The end-effector must lie on a plane perpendicular to this leg”

So:

- 3 legs $\rightarrow$ 3 planes
- Intersection of 3 planes $\rightarrow$ unique point

That's your FK.

If you're working on a *parallel-axis Tripteron specifically*

You likely have:

- All revolute axes parallel
- Linear rails or equivalent
- Parallelogram linkages enforcing orientation

👉 This simplifies things EVEN MORE:

- Orientation is fixed
- Only position matters

Suggested derivation path (what I'd do)

1. Define:
 - Base frame
 - End-effector point $p = [x, y, z]$
2. For each leg:
 - Write loop-closure equation
 - Use geometry → eliminate passive joints
3. Reduce each leg to:

$$n_i^T(p - p_i) = d_i(q_i)$$

4. Stack all 3:

$$Ap = b(q)$$

5. Solve:

$$p = A^{-1}b$$

If you want, I can go further

I can help you:

- Derive **full kinematics** for your exact geometry
- Map it to **Jacobian** (for force control / haptics)
- Compare it to **delta / orthoglide** mathematically
- Help implement it in **MATLAB** or **Python**

Just send:

- A sketch or CAD
- Joint definitions (PRRR, axes directions, etc.)

And we'll build it step-by-step.

     ...  Sources

I am working to make a tripteron where ALL of the axes are parallel to one another, not where they lie in different planes and directions as in the papers you linked me.

How do I go about this?

Thought for 17s >

Yes — and this changes the problem a lot.

What you are describing is **not the classical Tripteron geometry**. In the standard Tripteron, the three actuated prismatic joints are arranged so each leg contributes a different translational direction; that orthogonal arrangement is part of why the mechanism is fully decoupled and easy to model. Multiple Tripteron references explicitly describe the canonical mechanism that way. UFDC Images +2

So the first thing to do is mentally separate these two cases:

1. **Classical Tripteron:** 3 translational DOF, fixed orientation, nice decoupling.
2. **Your architecture:** all joint axes parallel, which means you cannot just reuse the usual Tripteron kinematic equations. You have to do a fresh **mobility and constraint analysis** first. Papers on translational parallel manipulators commonly do this using screw theory / constraint analysis before writing FK and IK, because the geometry alone can easily produce unintended rotation or overconstraint. Wiley Online Lib... +2

The key question

If **all the revolute/prismatic axes are parallel**, then a naive 3-leg build often does **not** automatically give pure 3T motion. More often, it tends to permit some combination of:

- $2T+1R$,

- 1T+2R,
- or a heavily overconstrained architecture,

unless each leg is arranged so that the **constraint wrenches** collectively kill all platform rotation while still leaving exactly three independent translations. That is the real design task. Wiley Online Lib... +2

How to go about it

Use this order:

1) Stop thinking “derive equations” first; do mobility synthesis first

For your concept, the first pass should be:

- choose one leg architecture,
- determine what motion that single leg allows the platform attachment point to have,
- determine what motions it constrains,
- intersect the allowed motions from all 3 legs.

This is exactly the style used in translational PM design papers: first identify the platform motion set, then write geometry. Wiley Online Lib... +1

In practice, ask of each leg:

- Does this leg allow the platform to translate in 3 directions?
- Does it forbid one or more rotations?
- Are those forbidden rotations from the 3 legs independent?

If the three legs do not collectively remove all 3 platform rotations, then you do **not** have a pure translational machine.

2) Model each leg as an allowed-twist / constraint-wrench object

This is the cleanest way to handle “all axes parallel.”

For the moving platform, write its instantaneous twist as

$$\mathcal{V} = \begin{bmatrix} \omega \\ v \end{bmatrix}$$

where ω is platform angular velocity and v is linear velocity.

For each leg, determine the platform twists compatible with the passive joints in that leg. Equivalently, determine the **constraint screws/wrenches** the leg applies. Then intersect all legs' twist spaces. This screw-theoretic approach is standard for analyzing translational parallel manipulators and is especially useful when geometry is nonstandard.

Wiley Online Lib... +2

Your goal is for the final platform twist space to be exactly

$$\omega = 0, \quad v \in \mathbb{R}^3$$

That is, the platform can translate arbitrarily, but cannot rotate.

3) Decide whether your platform orientation is enforced by parallelograms or by leg geometry alone

In many translational PMs, fixed orientation is not "free"; it is enforced by parallelogram subchains or equivalent constraint structures. Recent TPM papers explicitly note that the number and arrangement of parallelograms strongly affect structural complexity and decoupling. [arXiv](#) +1

So for your parallel-axis concept, ask:

- Are you using a parallelogram in each leg?
- Are the parallelogram planes all identical or offset?
- Are you relying on the 3 legs together to suppress rotation, or does each leg individually suppress some rotation?

If you are **not** using parallelograms or some equivalent orientation-locking feature, there is a good chance the platform will pick up an unwanted rotational DOF.

4) Once mobility is verified, write the kinematics as constraint equations, not borrowed Tripteron formulas

For your machine, inverse kinematics will likely come from loop-closure constraints of the form

$$\|p + r_i - a_i(q_i, \theta_i, \dots)\|^2 = L_i^2$$

or from line/plane constraints, depending on your leg architecture.

The procedure is:

- define a base frame,
- define platform reference point $p = [x \ y \ z]^T$,
- define platform orientation R ,
- write each leg closure equation,
- use the mechanism constraints to show that R must be constant,
- reduce the equations to a pure function of p and actuator variables.

If you cannot eliminate R , that is a sign your mechanism is not purely translational.

This “geometry only after mobility” approach is how the broader translational PM literature handles noncanonical architectures. ASME Digital Co... +2

A practical way to test your idea before building

Take your proposed leg design and do this on paper.

Let the platform have a small virtual motion $(\delta\omega, \delta p)$. For each leg, ask what infinitesimal motions are possible without changing any rigid-link lengths or violating joint directions.

Then stack those constraints into a matrix:

$$A \begin{bmatrix} \delta\omega \\ \delta p \end{bmatrix} = 0$$

You want the nullspace of A to be exactly 3-dimensional and spanned only by translations.

In other words, the nullspace basis should look like

$$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

after a coordinate change.

If instead a null vector contains angular terms, your platform can rotate.

What papers are still useful for you

Even though the standard Tripteron papers are not your geometry, they are still useful for:

- understanding how 3T parallel manipulators are framed,
- seeing how overconstraint is handled,
- seeing how inverse/direct kinematics are organized. [miscj.aut.ac.ir](#) +1

For **your** design process, the more useful reading is actually the broader translational-parallel-manipulator and screw-theory literature:

- mobility analysis of fully translational PMs, [Wiley Online Lib...](#)
- position analysis of new 3-DOF translational PM families, [ASME Digital Co...](#)
- newer TPM synthesis papers that explicitly derive motion/constraint structure before kinematics. [arXiv](#)

My blunt recommendation

Do **not** start by trying to write “the kinematics of a parallel-axis Tripteron.”

Start by answering:

- What is one leg’s exact joint sequence?
- Which joints are actuated?
- Are the passive axes all parallel too, or just some of them?
- Is there a parallelogram in each leg?
- How are the three leg attachment points arranged on the platform?

Because with all axes parallel, the architecture can easily stop being a 3T machine at all.

If you want, send me a quick sketch with:

- base attachment points,
- platform attachment points,
- joint type sequence in one leg,
- which direction all the parallel axes point,

and I’ll help you do the mobility test first, then derive the actual kinematics from that.

     ...    Sources

The parallelogram plnaes are identical. 2 of the translation stages have a single leg, and the 3rd translation stage has 2 legs

Thought for 44s >

That description is enough to say something useful: **your mechanism is probably not a “Tripteron-like 3T machine” by default anymore.** With all parallelogram planes identical, the legs tend to contribute **non-independent orientation constraints**. In translational parallel manipulators, getting pure translation depends on the legs providing the *right number of independent constraints*; papers on TPM synthesis and mobility analysis make that the first step, before any FK/IK derivation. ResearchGate +2

The big warning sign is this part:

- two actuators drive **single legs**
- one actuator drives **two legs**
- all parallelogram planes are **the same**

That usually means the duplicated pair is being used to add a missing rotational constraint. But if all four legs have the **same parallelogram-plane orientation**, then many of those constraints are parallel/redundant rather than independent. In other words, the fourth leg may add stiffness, but it may **not** add the “new kind” of constraint needed to guarantee pure 3D translation across the whole workspace. Broad TPM references emphasize that pure translation comes from the structure of the limb constraint set, not just from adding more legs. ResearchGate +2

So the right way to go about this is:

1. Treat it as a mobility problem first

Do **not** start with inverse kinematics. Start by asking:

What instantaneous platform twists $\begin{bmatrix} \omega \\ v \end{bmatrix}$ are allowed by my four legs?

For a true 3T platform, the only allowed twists should be

$$\omega = 0, \quad v \in \mathbb{R}^3$$

That is, no angular velocity at all, and three independent translations. This is exactly the style used in TPM type-synthesis and screw-theory papers. ResearchGate +1

2. Expect one of these two outcomes

With your architecture, the mechanism often falls into one of these buckets:

- **2T1R or 1T2R parasitic motion** somewhere in the workspace, because the rotational constraints are not independent enough.
- **Overconstraint**, where the extra fourth leg fights assembly tolerances or causes binding unless everything is extremely compliant/precise.

That risk is well known in lower-mobility parallel manipulators and translational PMs; unexpected rotations and overconstraint are exactly why designers analyze the limb wrench/constraint system first. [ResearchGate](#) +1

3. Your “two legs on one stage” is the place to inspect

Most likely, that double-leg stage is trying to do one of two things:

- make a single moving carriage behave like a **yoke**, resisting platform yaw/roll by spacing two attachment points apart, or
- emulate a **non-identical constraint family** using two copies of the same limb.

The first can help structurally. The second usually does **not** fully replace genuinely different limb orientations.

A practical way to think about it:

- if the two legs on the doubled stage are attached to the platform at **two distinct points**, then a relative platform rotation causes different length/velocity demands in those two legs
- that can suppress **some** rotation(s)
- but unless the other two single legs suppress the remaining rotation(s) independently, you still do not get guaranteed pure 3T motion

So the real question is not “do I have four legs?” but:

Do those four legs generate **three independent rotational constraints** on the moving platform?

If not, you do not have a pure translational mechanism.

4. The quickest paper-backed test

Build the **instantaneous constraint matrix** numerically for a guessed platform pose.

Let the platform twist be

$$t = [\omega_x \quad \omega_y \quad \omega_z \quad v_x \quad v_y \quad v_z]^T$$

For each leg, write one or more linear velocity-compatibility equations of the form

$$A_i t = b_i \dot{q}_i$$

or, if you eliminate actuator rates,

$$C_i t = 0$$

Then stack all legs:

$$C t = 0$$

Now inspect the nullspace of C .

You want:

- nullity = 3
- basis vectors corresponding to **pure translations only**

If the nullspace contains any vector with nonzero ω , then your platform can rotate.

If the nullity is less than 3, you are overconstrained.

5. My suspicion, based on your description

I would be suspicious that your current concept behaves like:

- **three translational commands near the center**, but
- with either a parasitic small rotation or a binding/overconstraint issue away from the nominal pose.

The reason is simple: the **classical Orthoglide/Tripteron-style TPMs deliberately use differently oriented limbs / actuator axes / parallelogram arrangements** so that the platform is restricted to pure translation. Orthoglide references explicitly note that the articulated parallelograms are oriented so the output body is restricted to translational motion only, and that the linear actuator axes are arranged orthogonally. arXiv +2

Your design removes that structural asymmetry, so you need to recreate the lost independent constraints some other way.

6. What I would do next

I would model it as a **4-leg translational PM**, not a Tripteron.

Use this workflow:

1. Pick a base frame.
2. Define the platform as a rigid body with pose (R, p) .

3. For each of the 4 legs, define:
 - base attachment point
 - platform attachment point
 - prismatic axis direction
 - parallelogram plane normal
4. Write the infinitesimal velocity of each platform attachment point:

$$v_i = v + \omega \times r_i$$

5. Project that velocity onto the directions each leg permits/forbids.
6. Stack the equations and compute the twist nullspace.

That will tell you, unambiguously, whether the machine is:

- true 3T,
- 2T1R,
- or overconstrained.

7. Blunt design advice

If your goal is **clean, predictable 3-axis translation**, then having **all parallelogram planes identical** is usually the first thing I would revisit. In most successful translational PM families, the limb constraints are arranged so that the three rotational constraints are clearly independent. arXiv +2

So the next design fork is:

- keep the architecture as-is and **prove by twist analysis** that it really is 3T, or
- intentionally redesign the limb orientations/attachment geometry so the rotational constraints are obviously independent.

If you send a sketch with:

- the common axis direction,
- the four platform attachment points,
- the four base attachment points,
- and which stage drives which leg(s),

I can help you write the actual constraint matrix and tell you whether the mechanism is 3T, 2T1R, or overconstrained.









 Sources

Ignore the above. I want to copy this design, ignoring the translation DOF. All I need is a 3 DOF parallel translation stage

Thought for 18s >

Yes — you can turn that paper's mechanism into a **3-DOF pure translation stage** by treating the rotational coordinate as **mechanically suppressed** and then simplifying the kinematics accordingly.

The paper's mechanism is a **4-DOF stage** with pose variables

$$(p_x, p_y, p_z, \phi)$$

where p is platform position and ϕ is rotation about the x -axis. The platform orientation matrix is written as a rotation about x , and all four leg-length constraints are then written in terms of p and ϕ .  Kinematic_Analysis_of_a_4-DOF_P...

What “ignoring the rotational DOF” means mathematically

For a 3T stage, just set the platform orientation to a constant:

$$\phi = \phi_0 \quad (\text{usually } 0)$$

Then the rotation matrix Q becomes constant rather than a variable. In the paper, the platform attachment points are written as

$$b_i = p + Qv_{i0}$$

so once Q is fixed, each Qv_{i0} is just a constant offset. That means the only unknowns left are p_x, p_y, p_z .  Kinematic_Analysis_of_a_4-DOF_P...

So the 4-DOF mechanism becomes a **3-DOF translational parallel mechanism** if you add hardware that prevents the x -axis rotation from happening.

The key design issue

In the paper, the mechanism has **two parallelogram legs**, and those parallelograms constrain **two rotational DOFs**, leaving **one rotation about x** still free. That remaining

rotation is not an accident; it is built into the architecture.  Kinematic_Analysis_of_a_4-DOF_P...

So if you want pure translation, you must add a way to eliminate that last rotation. In practice, that means one of these:

- change the leg architecture so the last rotational DOF is also constrained,
- add another parallelogram-based constraint,
- or use a duplicated/separated leg pair so the platform cannot twist about x .

So the job is not just “ignore ϕ in software.” You need to **remove that DOF physically**.

The cleanest way to reuse the paper’s model

Start from the paper’s position equations. The base points are

$$a_i = \rho_i e_y + \gamma_i e_x$$

and the platform points are

$$b_i = p + Q v_{i0}.$$

Each leg satisfies

$$(b_i - a_i)^T (b_i - a_i) = u_i^2.$$

Those are the core loop-closure equations.  Kinematic_Analysis_of_a_4-DOF_P...

For a 3T stage, set $Q = Q_0$ constant. Then define

$$c_i = Q_0 v_{i0},$$

which is now just a constant vector. Then

$$b_i = p + c_i.$$

So each leg becomes

$$(p + c_i - a_i)^T (p + c_i - a_i) = u_i^2.$$

Because $a_i = \rho_i e_y + \gamma_i e_x$, this is

$$(p + c_i - \rho_i e_y - \gamma_i e_x)^T (p + c_i - \rho_i e_y - \gamma_i e_x) = u_i^2.$$

That is your 3T loop-closure equation.

What the inverse kinematics looks like

If the actuator variables are the slider positions ρ_i , then for a given platform position p , each leg gives a quadratic in ρ_i , just like in the paper. The paper writes it as

$$\rho_i^2 + C_i \rho_i + D_i = 0$$

with coefficients built from the platform pose.  Kinematic_Analysis_of_a_4-DOF_P...

Once Q is fixed, those coefficients no longer depend on ϕ , only on p . So IK becomes:

- input: p_x, p_y, p_z
- output: $\rho_1, \rho_2, \rho_3, \rho_4$

This is the easiest direction.

What the velocity model becomes

The paper's velocity equation is

$$(b_i - a_i)^T (\dot{p} + \dot{\phi} EQv_{i0}) = \dot{\rho}_i (b_i - a_i)^T e_y.$$

If you remove the rotational DOF, then $\dot{\phi} = 0$, so this simplifies to

$$(b_i - a_i)^T \dot{p} = \dot{\rho}_i (b_i - a_i)^T e_y.$$

That is the translational Jacobian relation for your 3T stage.  Kinematic_Analysis_of_a_4-DOF_P...

So the platform velocity $\dot{p}$ is now related to the four slider rates by a **4-by-3** system instead of a 4-by-4 system.

What this tells you structurally

If you keep all four legs, your 3T stage is likely **redundantly constrained or redundantly actuated**, depending on how you implement it.

That is not necessarily bad. It can be good for stiffness. But it means:

- you cannot treat it like a minimal 3-leg Cartesian PM automatically,
- assembly tolerance matters,
- and the four constraints must be mutually compatible over the workspace.

The simplest practical interpretation

A very workable way to proceed is:

1. Use the paper's geometry as the starting point.  Kinematic_Analysis_of_a_4-DOF_P...
2. Freeze the platform orientation at $\phi = 0$.

3. Replace the 4-DOF pose (p_x, p_y, p_z, ϕ) with just (p_x, p_y, p_z) .
4. Keep the same leg closure equations, but with constant attachment offsets.
5. Check whether the resulting 4-leg system is compatible and nonsingular over the workspace.

What you should physically change

Because the original paper intentionally leaves one rotation free, you need to add a constraint that kills rotation about x . The most straightforward options are:

- convert one of the non-parallelogram legs into another parallelogram-constrained leg,
- use two separated legs on one side of the platform so that any x -rotation creates an incompatible differential motion,
- or add a passive guide that allows only translation of the end platform.

That last option is mechanically simplest for prototyping, though it partly defeats the “fully parallel” purity.

What to derive next

For your 3T version, the derivation order should be:

1. Choose a fixed platform orientation

Usually

$$Q_0 = I.$$

2. Define the constant platform offsets

$$c_i = v_{i0}.$$

3. Write the four leg equations

$$(p + c_i - \rho_i e_y - \gamma_i e_x)^T (p + c_i - \rho_i e_y - \gamma_i e_x) = u_i^2, \quad i = 1, \dots, 4.$$

4. Differentiate for Jacobian

$$(b_i - a_i)^T \dot{p} = \dot{\rho}_i (b_i - a_i)^T e_y.$$

5. Check singularity by rank loss

You now care about when the translational Jacobian loses rank, not the original 4-DOF determinant condition from the paper.

Bottom line

You can absolutely use that paper as the template for a **3-DOF parallel translation stage**, but the right interpretation is:

- keep the paper's loop-closure framework,
- set ϕ constant,
- simplify the Jacobian by setting $\dot{\phi} = 0$,
- and most importantly, **add a real mechanical constraint that eliminates the paper's remaining rotational DOF.**  Kinematic_Analysis_of_a_4-DOF_P...

If you want, I can take the paper's equations and rewrite them cleanly as a **3T-only kinematic model** in either MATLAB notation or symbolic form.



I don't need 4 legs, is there a way I can use the kinematics given in the paper to design a similar but distinct 3 DOF cartesian mechanism using only 3 legs?

Yes — but with one important caveat:

you can reuse the paper's kinematic style, but not just delete one leg from that architecture and expect a valid 3T mechanism.

That paper's 4-leg mechanism gets its motion and constraints from the combined action of all four limbs; two of the limbs are parallelogram-based and the platform pose is modeled with 3 translations plus 1 rotation about x . The loop-closure is written leg-by-leg as

$$(b_i - a_i)^T (b_i - a_i) = u_i^2,$$

with

$$a_i = \rho_i e_y + \gamma_i e_x, \quad b_i = p + Q v_{i0},$$

so the whole method is very reusable even if the architecture changes.

 Kinematic_Analysis_of_a_4-DOF_P...

The right takeaway is:

- reuse the modeling framework
- do not reuse the exact leg count/constraint structure

What a 3-leg version would need to be

If you want a **3-DOF Cartesian parallel mechanism with pure translation**, then the platform must have:

- 3 allowed translational DOFs
- 0 rotational DOFs

So your 3 legs must collectively remove all 3 platform rotations.

That means each leg cannot be “just a distance link” in the naive sense. A simple 3-strut platform does **not** automatically give pure translation. You need each limb to include geometry that contributes rotational constraints, usually by using **parallelograms** or an equivalent orientation-preserving subchain.

So the practical answer is:

A plausible 3-leg strategy

Use **three identical translational limbs**, each with a parallelogram, so that:

- each limb is driven by one linear actuator,
- the platform orientation is preserved by the three parallelograms together,
- and the end platform is left with only 3 translational DOFs.

Conceptually, this is much closer to a **Delta/Orthoglide-style 3T parallel robot** than to the paper’s 4-leg, 3T+1R mechanism.

How to reuse the paper’s equations anyway

The paper gives you a very nice template:

1. define base attachment points a_i ,
2. define platform attachment points b_i ,
3. impose constant leg lengths,
4. differentiate for the Jacobian.  Kinematic_Analysis_of_a_4-DOF_P...

You can do exactly that for a 3-leg mechanism.

For a pure translation stage, set the platform orientation constant. Then Q is fixed, so

$$b_i = p + c_i,$$

where c_i is a constant platform offset for leg i .

Then for **three legs**:

$$(p + c_i - a_i)^T (p + c_i - a_i) = u_i^2, \quad i = 1, 2, 3.$$

That is the direct analogue of the paper's equation, just with 3 legs instead of 4 and no platform rotation variable.

If your base sliders are still all parallel to y , as in the paper, then

$$a_i = \rho_i e_y + \gamma_i e_x.$$

Substituting gives

$$(p + c_i - \rho_i e_y - \gamma_i e_x)^T (p + c_i - \rho_i e_y - \gamma_i e_x) = u_i^2.$$

This is a perfectly valid starting point for **inverse kinematics**:

given p , solve for ρ_1, ρ_2, ρ_3 .

What this buys you

This formulation gives you a clean way to design geometry:

- c_i : where the legs attach on the moving platform
- γ_i : where the actuator lines sit on the base
- u_i : link lengths

You can choose these to shape workspace and avoid interference, just like the paper does with its designs.  [Kinematic_Analysis_of_a_4-DOF_P...](#)

The real design question: can 3 legs be enough?

Yes, if **each leg is a constraint-producing limb**, not just a point-to-point rod.

A 3-leg pure-translation mechanism is feasible when each leg contains something like a parallelogram that prevents the platform from freely rotating relative to that limb. Then the three limbs together leave translation only.

A good mental model is:

- each leg contributes one actuated scalar motion,
- each leg also contributes some passive geometry that helps suppress rotation,
- all three together must eliminate roll, pitch, and yaw.

If you do not build that into the limbs, then the platform will generally still be able to rotate.

A very practical architecture to consider

If your goal is “similar spirit to the paper” and all base actuators are parallel sliders, one reasonable architecture is:

- 3 parallel linear actuators on the base
- 3 limbs connecting those sliders to a rigid platform
- a parallelogram in each limb
- three distinct platform attachment points

Then model it exactly as above.

This will be a **distinct mechanism**, not just a trimmed version of the paper, but the kinematic derivation style is the same.

Velocity model

The paper differentiates the loop closure and gets a Jacobian relationship between platform velocity and actuator rates. For your 3T mechanism, with fixed orientation, the rotational term drops out and each leg gives

$$(b_i - a_i)^T \dot{p} = \dot{\rho}_i (b_i - a_i)^T e_y.$$

Stacking the 3 leg equations gives a 3×3 translational Jacobian:

$$J_p \dot{p} = K \dot{\rho}.$$

This is exactly the kind of clean Cartesian mapping you want in a 3-leg translational mechanism, and it comes directly from the same derivation pattern as the paper.

 Kinematic_Analysis_of_a_4-DOF_P...

The design workflow I would recommend

Use the paper’s modeling style, but do this:

Step 1

Assume a rigid platform with fixed orientation.

Step 2

Pick three platform attachment points c_1, c_2, c_3 .

Step 3

Pick three base actuator lines:

$$a_i = \rho_i e_y + \gamma_i e_x$$

or a generalized version if your base layout differs.

Step 4

Write

$$\|p + c_i - a_i\|^2 = u_i^2, \quad i = 1, 2, 3.$$

Step 5

Differentiate to get the Jacobian.

Step 6

Then do the critical check:

make sure your actual leg architecture physically enforces constant platform orientation.

That last part is the difference between “nice equations on paper” and a real 3T machine.

Bottom line

Yes, you can absolutely use the paper’s kinematic framework to design a **similar but distinct 3-leg Cartesian parallel mechanism**.

But the correct interpretation is:

- use the paper’s **loop-closure and Jacobian method**,
- reduce the pose to translation only,
- write **3 leg equations instead of 4**,
- and design each leg so the mechanism physically preserves platform orientation.

So the answer is **not** “remove one leg from the paper’s mechanism,” but rather:

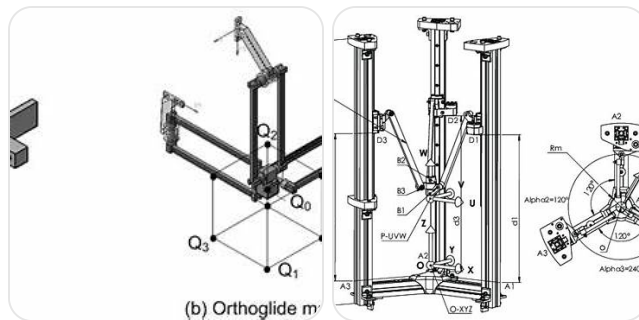
“build a new 3-leg pure-translation mechanism and model it using the same kinematic formalism the paper uses.”  Kinematic_Analysis_of_a_4-DOF_P...

Send me a sketch of the 3-leg concept you have in mind, and I can help you write the actual kinematic equations for that version.



What I like about this paper is that all DOFs are actuated from parallel linear stages. Can you help me find a derivation for a delta or orthoglide style mechanism with this property?

Thought for 46s >



Yes. The mechanism family you want is **not the classical rotary-actuated Delta**, but either a **linear Delta** or an **Orthoglide-type 3T manipulator**. Both are 3-DOF translational parallel robots whose motion is driven entirely from **linear stages**, and in the linear Delta case those stages can be arranged **parallel to one another**. The Orthoglide also uses only linear actuators, but its canonical version uses **three mutually orthogonal** linear guides rather than parallel ones. [arXiv +2](#)

For your goal, I would split the options like this:

- **Linear Delta:** best match if the feature you care about most is "all actuators are parallel linear stages." EPFL's linear-Delta work explicitly discusses Delta variants with **linear actuation** and presents that as a core architectural advantage. [ResearchGate](#)
- **Orthoglide:** best match if the feature you care about most is "clean Cartesian 3T motion with simple kinematics and fixed platform orientation." The Orthoglide papers describe it as a 3-DOF parallel mechanism with **three fixed linear joints** and a platform that moves in Cartesian x, y, z with **fixed orientation**. [arXiv +1](#)

The cleanest derivation to start from is probably the **Orthoglide**, because its constraint equations are unusually simple. In the standard Orthoglide model, the TCP position $p = (x, y, z)$ and actuator coordinates (ρ_x, ρ_y, ρ_z) satisfy three sphere equations of the form

$$(x - \rho_x)^2 + y^2 + z^2 = L^2, \quad x^2 + (y - \rho_y)^2 + z^2 = L^2, \quad x^2 + y^2 + (z - \rho_z)^2 = L^2,$$

where L is the constant leg length. The Orthoglide literature uses this model to derive both inverse and direct kinematics, workspace, and singularity conditions. [arXiv +1](#)

If you want **parallel linear stages**, though, the **linear Delta** is architecturally closer to your paper. A useful starting reference is *The Linear Delta: Developments and Applications*, which focuses specifically on Delta structures using **linear actuation** rather than rotary actuation. [ResearchGate](#) Another accessible derivation-oriented reference is *A Novel Inverse Kinematic Approach for Delta Parallel Robot*, which explicitly treats a **linear delta robot** and discusses inverse kinematics for that architecture. [ijmmm.org](#)

So the practical recommendation is:

- If you want the **easiest math**, start with **Orthoglide derivations**. The main paper to read is “**The Orthoglide: Kinematics and Workspace Analysis**”. [arXiv](#)
- If you want the **closest architecture** to “three parallel sliders driving a 3T parallel stage,” start with **linear Delta papers**, especially “**The Linear Delta: Developments and Applications**” and then a derivation paper like the inverse-kinematics paper above. [ResearchGate +1](#)

A subtle but important point: if you try to make an “Orthoglide-like” machine while keeping **all three linear stages parallel**, you stop having the canonical Orthoglide geometry. At that point the mechanism becomes closer to a **linear Delta / translational PKM with parallel actuators**, and you need to redo the constraint geometry instead of directly copying Orthoglide equations. The Orthoglide’s simplicity depends strongly on its **orthogonal** actuator arrangement. [arXiv +1](#)

A good reading order would be:

1. **Orthoglide: Kinematics and Workspace Analysis** — to learn the clean 3T derivation style. [arXiv](#)
2. **Kinematic Analysis of a New Parallel Machine Tool: The Orthoglide** — same family, more design context. [arXiv](#)
3. **The Linear Delta: Developments and Applications** — to switch from orthogonal linears to the all-linear-actuated Delta family. [ResearchGate](#)
4. **A Novel Inverse Kinematic Approach for Delta Parallel Robot** — for a more direct linear-Delta IK treatment. [ijmmm.org](#)

My honest recommendation is: **derive from Orthoglide to learn the math, then adapt a linear Delta for your actual hardware concept**. Orthoglide is easier to understand

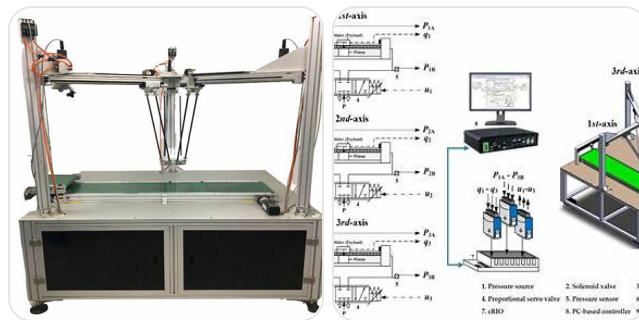
analytically; linear Delta is closer to the “all DOFs actuated from parallel linear stages” feature you want. [arXiv +1](#)

I can also walk you through an actual derivation for either one, starting from the loop-closure equations and ending with IK/Jacobian.

        Sources

A horizontal linear delta actually seems to be the best solution for my requirements

Thought for 8s >



I agree — a **horizontal linear delta** sounds like a strong fit for what you want.

Why it matches well:

- it keeps **all three actuators as linear stages**, and they can be arranged **parallel to each other**, which is the property you liked in the 4-DOF paper;
- it is still a **3-DOF translational parallel mechanism**, so you avoid carrying an extra rotational DOF you do not want;
- there are published examples specifically on **horizontal-structure linear delta robots** using three horizontal actuators, plus broader linear-delta papers that cover the kinematic ideas behind the family. [MDPI +1](#)

The main tradeoff is that a horizontal linear delta is usually **less algebraically clean than an Orthoglide**. Orthoglide kinematics are famous for their simple sphere equations, but that simplicity depends on the canonical orthogonal guide layout. A horizontal linear delta is closer to your preferred hardware architecture, but the geometry is a bit less decoupled.

[arXiv +1](#)

So I would frame the next step like this:

- Use **Orthoglide** papers to learn the derivation style.
- Use **linear-delta** papers to choose the actual architecture.
- Then write the kinematics for your specific horizontal layout.

A good set of starting references is:

- **The Linear Delta: Developments and Applications** for the architecture family and why linear actuation is attractive. [ResearchGate](#) +1
- **Development of a Linear Delta Robot with Three Horizontal-Axial Pneumatic Actuators** for a directly relevant horizontal implementation. [MDPI](#)
- **The Orthoglide: Kinematics and Workspace Analysis** for a very clean 3T derivation template. [arXiv](#) +1

The practical design recipe I'd suggest is:

1. pick three parallel guide lines;
2. place one carriage on each guide;
3. connect each carriage to the moving platform through a delta-style limb with a parallelogram or equivalent orientation-preserving linkage;
4. write one loop-closure equation per leg;
5. solve inverse kinematics first, then Jacobian, then workspace/singularity conditions.

That gives you the architecture you want without forcing yourself into the 4-leg mechanism from the paper.

The next useful thing would be to derive a **minimal symbolic model** for a horizontal linear delta in your preferred geometry. I can do that with you from scratch and keep it as simple as possible.

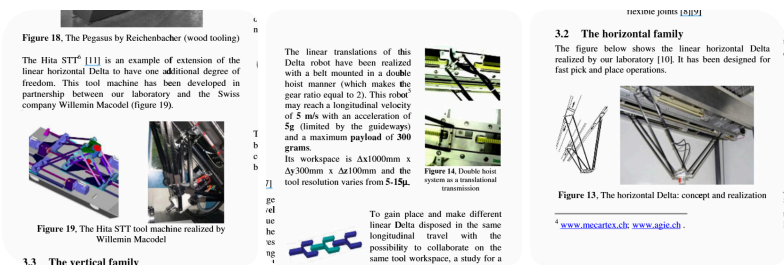








Sources



R., "A general overview of parallel kinematics", 1. Internationales Symposium IPK 98, pp. 9-24.
 .., Méthode systématique de la cinématique des parallèles. PhD Thesis, 2004.

These are the kinds of linear deltas I'm interested in. They have 3 PARALLEL, HORIZONTAL AXES

Great — now we're locked onto the **exact architecture** you want 👍

This is the **horizontal linear delta family** (EPFL / Clavel lineage), and it's actually a really nice middle ground between:

- the clean math of Orthoglide
- and the "all actuators are parallel linear stages" constraint you care about

🧠 First: what this mechanism *really* is

Horizontal Linear Delta (your images)



Key structure

- 3 **parallel linear guides** (all along the same direction, say y)
- 3 moving carriages
- Each carriage connects via a **delta limb**:
 - upper link (rigid)
 - parallelogram (or equivalent) → enforces **constant platform orientation**
- End-effector = intersection of 3 spherical constraints

- 👉 This is **NOT** an Orthoglide
- 👉 This is **NOT** the 4-leg paper

It's a 3-legged 3T parallel manipulator with parallel actuation

🔑 The key modeling insight (this is the breakthrough)

Even though all actuators move along **the same direction**, the mechanism still achieves 3D motion because:

Each leg constrains the end-effector to lie on a **sphere centered on the carriage**

So instead of planes (Tripteron) or orthogonal spheres (Orthoglide), you get:

- 👉 3 moving sphere centers sliding along parallel lines
-

🧮 Clean kinematic derivation (tailored to your design)

Let's build the **minimal model** you actually want.

1. Define coordinates

Let:

- End effector position:

$$p = (x, y, z)$$

- Actuator positions:

$$\rho_1, \rho_2, \rho_3$$

- All actuators move along y

So base points:

$$a_i = (x_i, \rho_i, z_i)$$

where:

- x_i, z_i = fixed rail positions
- ρ_i = slider position

2. Platform attachment points

Because of parallelograms:

- 👉 orientation is fixed
- 👉 offsets are constant

So:

$$b_i = p + c_i$$

where c_i are fixed vectors

3. Leg length constraints (THIS is the core)

Each leg has fixed length L_i :

$$\|b_i - a_i\|^2 = L_i^2$$

Substitute:

$$\|p + c_i - a_i\|^2 = L_i^2$$

Expand:

$$(x + c_{ix} - x_i)^2 + (y + c_{iy} - \rho_i)^2 + (z + c_{iz} - z_i)^2 = L_i^2$$

💥 This is your full inverse kinematics

Given $p = (x, y, z)$, solve for ρ_i :

$$\rho_i = y + c_{iy} \pm \sqrt{L_i^2 - (x + c_{ix} - x_i)^2 - (z + c_{iz} - z_i)^2}$$

Interpretation (very important)

- Each actuator controls **distance along y**
- But motion in x, z comes from geometry of sphere intersections
- This is why parallel actuators still give 3D motion



Forward kinematics (harder, like all deltas)

You now have 3 equations:

$$(x + c_{ix} - x_i)^2 + (y + c_{iy} - \rho_i)^2 + (z + c_{iz} - z_i)^2 = L_i^2$$

Solve for x, y, z .



This is a **3-sphere intersection problem**



Typically solved by:

- subtracting equations pairwise (eliminate squares)
- reducing to linear + one quadratic



Jacobian (for your haptics / force work)

Differentiate constraint:

$$(b_i - a_i)^T \dot{p} = (b_i - a_i)^T e_y \dot{\rho}_i$$

Stack $\rightarrow$ matrix form:

$$J_p \dot{p} = K \dot{\rho}$$

Where:

- rows of $J_p = (b_i - a_i)^T$
- $K = \text{diag}((b_i - a_i)^T e_y)$



This is **identical structure to the paper you liked**



but now for a 3-leg system



Why this architecture works (intuitively)

Each leg says:

"The end effector must stay a fixed distance from my carriage"

So:

- 1 leg $\rightarrow$ sphere
- 2 legs $\rightarrow$ circle
- 3 legs $\rightarrow$ point

And because:

- the sphere centers slide along **parallel lines**
- the intersection still spans 3D

Design pitfalls (important for you)

1. Singularity

≡ ChatGPT ▾

↑ ...

- the three leg direction vectors become coplanar
- or two spheres become tangent

→ Jacobian loses rank

2. Workspace shape

Unlike Orthoglide:

- workspace is **curved / dome-like**
- not cubic

3. Sensitivity to geometry

Your choices of:

- x_i, z_i (rail spacing)
- c_i (platform geometry)
- L_i

👉 strongly affect:

- isotropy
- stiffness
- singularities

Bottom line

You've basically landed on:

The only 3DOF parallel mechanism that has:

- “all actuators linear”
- “all actuators parallel”
- “full 3D translation”

That’s why this design shows up in high-speed pick-and-place machines.

If you want next steps

I can help you:

1. **Pick optimal geometry** (rail spacing, link lengths)
2. Derive **closed-form FK**
3. Compute **Jacobian + force mapping** (important for your haptics work)
4. Help design a **CAD-ready layout**

Just tell me 



Can I save the contents of our conversation as a PDF?

Yes — there are a few easy ways to save this conversation as a PDF depending on how you’re using ChatGPT:

Option 1: Use your browser (simplest)

1. Press **Ctrl + P** (Windows) or **Cmd + P** (Mac)
2. In the printer options, choose “**Save as PDF**”
3. Adjust settings if needed:
 - Layout: Portrait
 - Margins: Default or Minimal
4. Click **Save**

 This will capture the full conversation as it appears on screen.

✅ Option 2: Copy → Paste into a document

If you want cleaner formatting:

1. Select the conversation text
2. Copy it
3. Paste into:
 - Word / Google Docs / Notion
4. Export as PDF

👉 Better if you want to:

- edit
 - add notes
 - clean formatting
-

✅ Option 3: Export (if available in your UI)

Some ChatGPT interfaces have:

- “Share” → “Export” → PDF

If you see a share button at the top, check there.

⚠️ Tips for best results

- Expand all messages before printing (scroll through everything)
 - Zoom out slightly (90–95%) to avoid weird page breaks
 - If images are included, Option 2 usually formats them better
-

If you want, I can also **reformat everything we discussed into a clean PDF-style writeup** (like lecture notes or a design document).



